ACTIVITY 2

NLP-Based Intelligent Writing Assistant

Autocorrect, Autocomplete, and POS-Aware Suggestions

teh → **the**

jumpd → **jumped**

ovr → **over**

Spell correction · next-word prediction · POS tagging — built from scratch in Python

TEAM

SyntaxSquad

MEMBERS

Anoon M R · 25MCAR0179

Aman Kumawat · 25MCAR0023

Adithya Krishnan T N · 25MCAR0003

FACULTY

Dr. Sharal Coelho

COURSE

Natural Language Processing (25MCAAI303)

DEPARTMENT

MCA · JAIN (Deemed-to-be University)

Introduction & Problem Statement

Background

A writing assistant built from scratch in Python — spell correction, next-word prediction and POS-aware suggestions.

Problem

Spell checkers ignore real error patterns; autocomplete ignores grammar.

Why it matters

Emails, messages and academic writing all suffer from context-blind corrections and ungrammatical suggestions.

Objectives — five modules

1. **Tokenizer**
   Regex-based word splitting
2. **Spell corrector**
   Noisy Channel, learned confusion matrix, edit distance ≤ 2
3. **N-Gram LM**
   Bigram + trigram with back-off
4. **POS tagger**
   HMM decoded with Viterbi
5. **Live Flask UI**
   Debounced AJAX, updates as you type

Literature Survey



CACM

Damerau

~80% of real typos are a single edit

IEEE Trans. Info. Theory

Viterbi

The decoding algorithm our POS tagger uses

ANLP

Church

HMM-style POS tagger on the Brown Corpus, 95–99% accuracy

COLING

Kernighan, Church & Gale

Noisy Channel spelling correction with learned confusion matrices — basis of our corrector

CSL

Chen & Goodman

Smoothing survey; explains our perplexity results

Research gap: these techniques are usually studied in isolation — we let the HMM filter the N-gram LM's suggestions in real time.

Proposed Methodology



Spell corrector

Candidates within edit distance 2, ranked by P(word) × P(typo | word). P(typo | word) learned from 12,356 real single-edit misspellings.

N-Gram language model

Trigram with back-off to bigram, Laplace smoothing; trained on Brown Corpus sentences.

HMM POS tagger

Bigram HMM over 10 Universal POS tags, decoded with the Viterbi algorithm.

POS-aware filter

Keep only next-word candidates whose tag matches the tag the HMM expects next.

Dataset & Experimental Setup

54,286
cleaned sentences
Brown Corpus (NLTK)

24,891
vocabulary words
frequency ≥ 2

54,270
POS-tagged sentences
10 Universal tags

38,218
real misspelling pairs
Norvig corpus, 7,841 words

Downloaded from: **Brown Corpus** — NLTK data (nltk.download('brown'), nltk.org/nltk_data) **Misspellings** — norvig.com/ngrams/spell-errors.txt

Preprocessing (Activity 1)

1. Lowercase
2. Strip punctuation
3. Tokenize
4. Length filter

Experimental setup



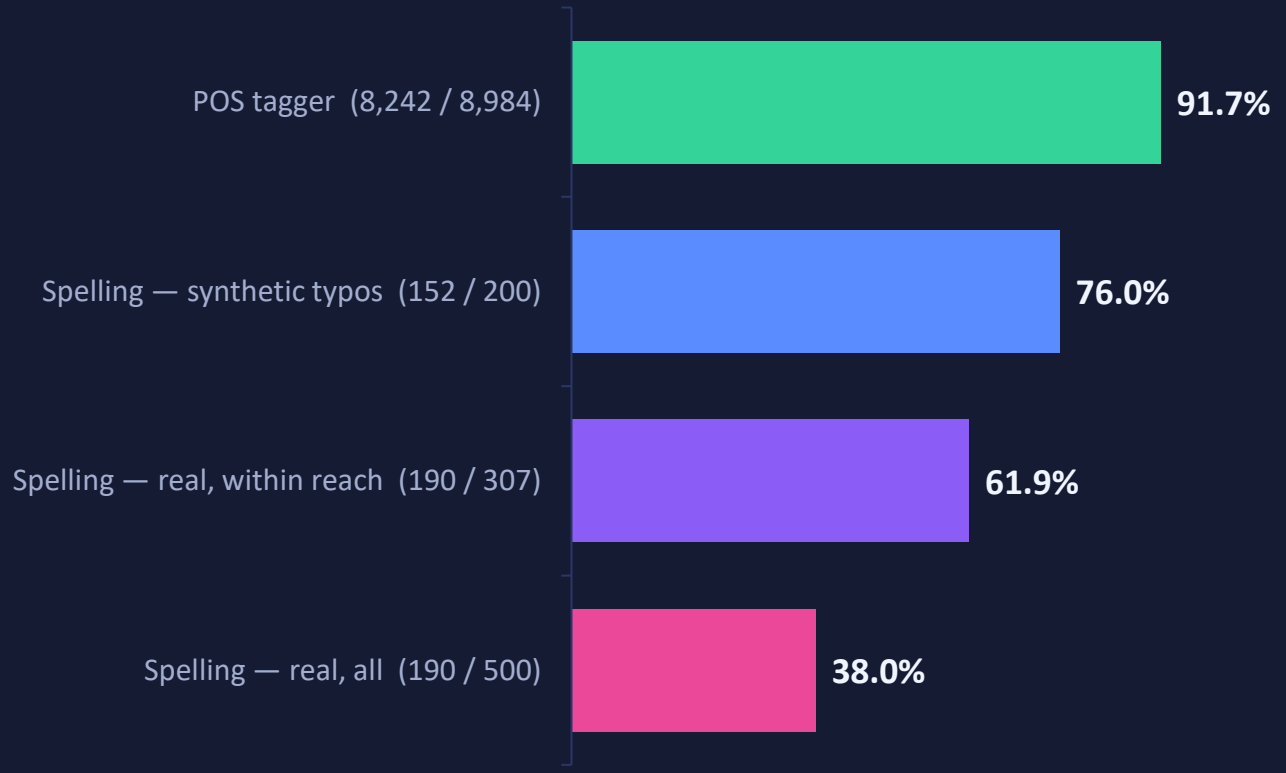
seed 42

Applied to all three datasets — every score is on unseen data.

Tools: Python 3.12 · Flask 3.1.3 · matplotlib · no GPU

Results & Evaluation

Accuracy on held-out test data



5,596

Bigram perplexity

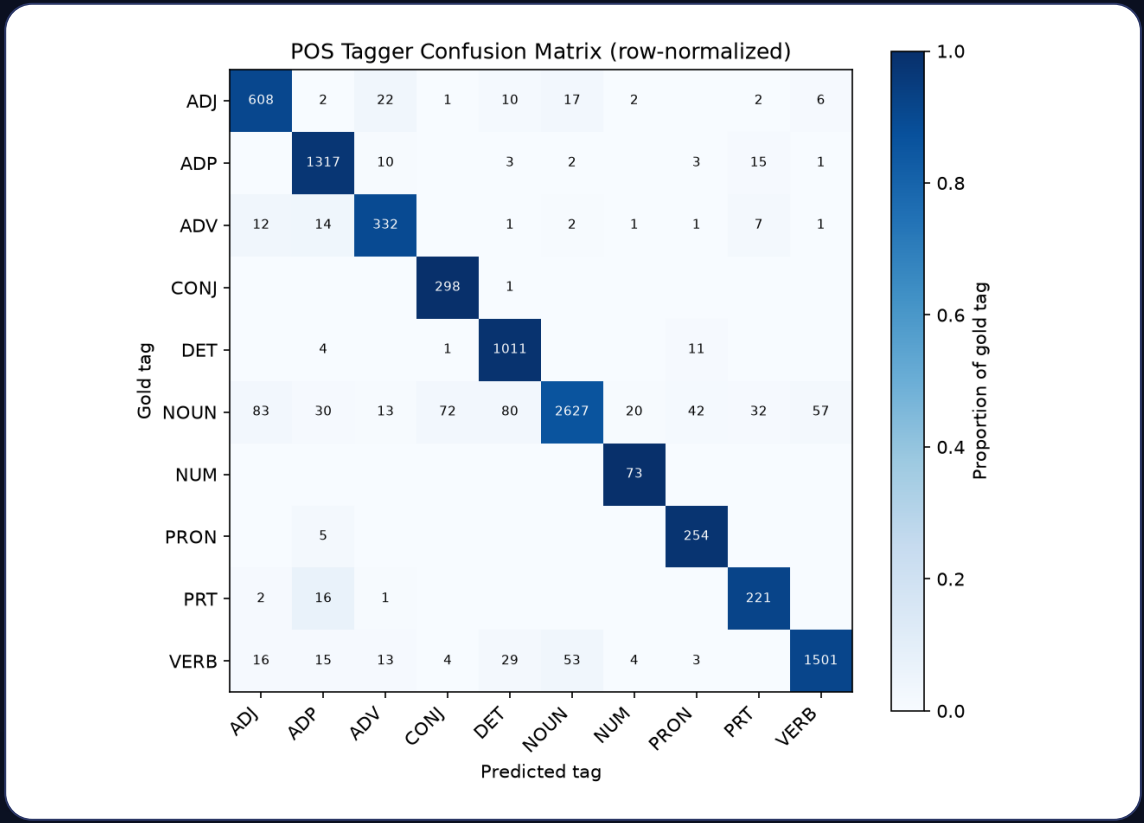
13,641

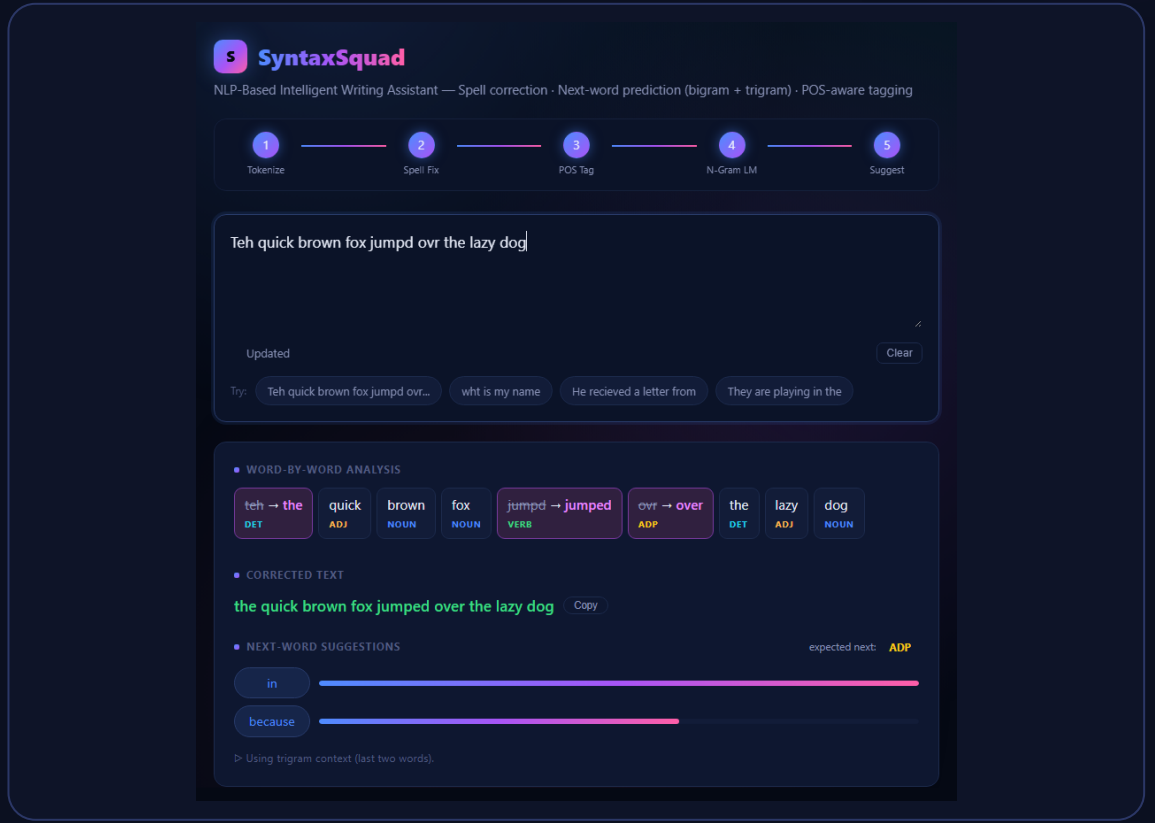
Trigram perplexity (with back-off)

Why is trigram higher? Laplace adds V ≈ 40K to every denominator, which swamps the sparse trigram counts.

193 of 500 real misspellings are 3+ edits away — out of reach by design.

Confusion Matrix & Live Output



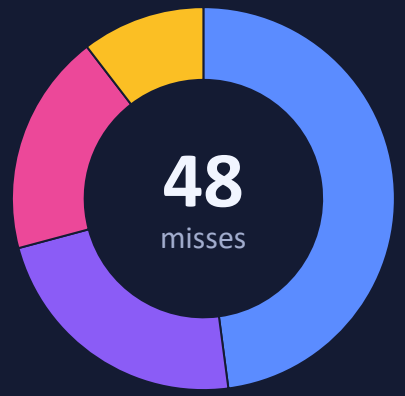


L **POS tagger confusion matrix**
Held-out set, strongly diagonal. Biggest confusions: NOUN → ADJ (83), DET (80), CONJ (72).

R **Live web app**
3 typos fixed, every word tagged, next-word suggestions filtered to the expected tag.

Error Analysis

Why the 48 synthetic-typo misses happened



- **23** ED-2 word beat ED-1
- **11** Other ED-1 word won
- **9** Real-word typo
- **5** Not in vocabulary

haed → “and”, not “head”: a very common word 2 edits away outranks the right word 1 edit away.

will → gill: a real word, so words corrected in isolation never flag it.

Fixed during testing

“ovr” used to give “or” and “wht” gave “who” — both now give “over” and “what”, thanks to the learned confusion matrix. Single-letter “I” → “in” also fixed.

POS confusions

NOUN mix-ups on ambiguous words like “that” — a bigram HMM only sees one previous tag.

Next fixes

Prefer 1-edit candidates · rank spelling in context · Kneser-Ney smoothing · trigram HMM.

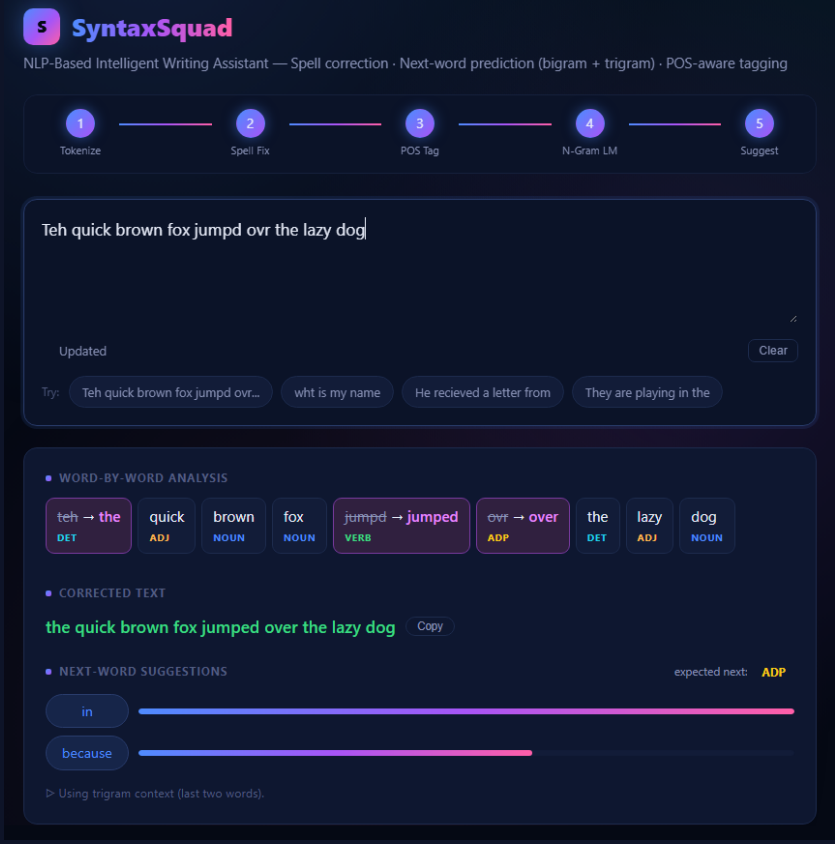
Project Proof

Source code on GitHub

github.com/AnoonMR/Writing-Assistant

```
$ pip install -r requirements.txt
$ cd src
$ python app.py
→ open http://127.0.0.1:5000
$ python evaluate.py   # reproduce every metric
```

Not publicly hosted — runs on localhost, fully offline once the data is downloaded.



Conclusion & Future Work

5 / 5

modules built

At full proposal scope, integrated in one live web app and evaluated on held-out data.

91.74%

POS tagging accuracy

Spelling: 76.00% on synthetic typos and 61.9% on real misspellings within reach — all from scratch.

3

weak spots, all diagnosed

Each has a measured cause: frequency bias at edit distance 2, out-of-range misspellings, Laplace smoothing.

Future work

- Prefer 1-edit candidates
- Context-aware spelling
- Kneser-Ney smoothing
- Trigram HMM
- Optional hosting

AI Usage Acknowledgement



This project was developed with the assistance of Claude (Anthropic), an AI tool, which was used for code implementation and debugging.

References

1. Damerau, F. J. (1964). A technique for computer detection and correction of spelling errors. CACM, 7(3).
2. Kernighan, M. D., Church, K. W., & Gale, W. A. (1990). A spelling correction program based on a noisy channel model. COLING.
3. Katz, S. M. (1987). Estimation of probabilities from sparse data... IEEE Trans. ASSP, 35(3).
4. Brown, P. F. et al. (1992). Class-based n-gram models of natural language. Computational Linguistics, 18(4).
5. Chen, S. F., & Goodman, J. (1999). An empirical study of smoothing techniques for language modeling. CSL, 13(4).
6. Church, K. W. (1988). A stochastic parts program and noun phrase parser for unrestricted text. ANLP.
7. Viterbi, A. J. (1967). Error bounds for convolutional codes... IEEE Trans. Information Theory, 13(2).
8. Norvig, P. (2009). Natural language corpus data. In Beautiful Data, O'Reilly, 219–242.
9. Brants, T. (2000). TnT — A statistical part-of-speech tagger. ANLP.